AI Agriculture Climate Planner

Time plan

Part	Topic	Time
1	Introduction	1 min
2	The problem	1.5 min
3	Solution and features	1.5 min
4	Architecture overview	2 min
5	Why multi-agent with LangGraph	2 min
6	The agents one by one	3 min
7	RAG pipeline	3 min
8	Machine learning part	1.5 min
9	Multilingual support	1 min
10	Caching and API optimisation	2 min
11	Streamlit interface	1 min
12	Demo walk-through	1 min
13	Challenges	1.5 min
14	Limitations and future work	1.5 min
15	Closing	1 min
	Total	**about 24 min**

Speaking speed: about 130 words per minute. Do not memorise word by word. Memorise the order of the parts and the main idea of each one.

THE SCRIPT

Part 1: Introduction (1 min)

Good morning. My name is [Your Name]. I am a final-year B.Tech student in Artificial Intelligence and Data Science.

Today I will explain my project, **AI Agriculture Climate Planner**.

It is an AI assistant for farmers. A farmer can ask questions about which crop to grow, how to treat a plant disease, when to water the field, and how weather risks may affect the farm. The system gives clear, practical answers, and it can reply in the farmer's own language.

I built it with **Python, Streamlit, LangGraph, LangChain, Groq, Hugging Face models, and FAISS**.

Part 2: The problem (1.5 min)

Let me explain the problem first.

Farming in India depends heavily on weather. But the climate is becoming less predictable. Rain comes late. Heat waves are stronger. Dry periods are longer.

Farmers must take many decisions. Which crop is right for this soil and this season? When should I irrigate? Is this spot on my leaf a disease? Will the next two weeks bring too much rain or too little?

Today, a farmer's information is often scattered. Some comes from neighbours, some from the weather app, some from the shop owner, and some from the agriculture office. It is not always easy to reach, and many guides are in English.

There is also a problem with normal AI chatbots. A general chatbot can give an answer that sounds good but is wrong. In farming, a wrong answer can destroy a whole crop.

So my question was: **can we build one assistant that gives reliable, local, and easy-to-understand farming advice, in the farmer's language?**

Part 3: Solution and features (1.5 min)

My solution is a single web app with six main features.

1. **Crop planning:** suggests suitable crops using soil and climate details.
2. **Disease prediction:** helps identify a likely plant disease and gives treatment advice.
3. **Irrigation scheduling:** tells when and how much to water.
4. **Soil analysis:** reads soil values like nitrogen, phosphorus, potassium, and pH, and suggests how to improve the soil.

5. **Climate risk forecasting:** warns about heat, drought, or heavy rain risk.
6. **Multilingual AI assistant:** answers questions in English, Hindi, and Telugu.

Behind these features, I used a **multi-agent generative AI design** with **RAG** and **machine learning**. I also added **caching**, to reduce the number of calls to the LLM.

Part 4: Architecture overview (2 min)

Let me explain the architecture in simple steps.

Step 1: User input. The farmer opens the Streamlit app, selects a language, and types a question. They can also enter details like location, crop, and soil values.

Step 2: Cache check. The system first checks if the same question was already answered. If yes, it returns the saved answer immediately.

Step 3: Router agent. If it is a new question, a router agent reads it and decides what type of question it is. Is it about crops, disease, irrigation, soil, or climate?

Step 4: Specialist agent. The question goes to the right specialist agent. Each agent has its own job and its own tools.

Step 5: Data and knowledge. The agent may fetch weather data, call a machine learning model, or search my agriculture knowledge base using RAG.

Step 6: LLM answer. The agent sends the collected facts to the **Groq LLM**, which writes a clear answer in the user's language.

Step 7: Response and caching. The answer is shown in Streamlit and saved in the cache.

So the LLM does not work alone. It is supported by **real data, a knowledge base, and ML models**.

Part 5: Why multi-agent and why LangGraph (2 min)

Why did I not use one big prompt for everything?

A single prompt that handles crops, disease, irrigation, soil, and climate becomes long and confusing. The model makes more mistakes, and it is hard to fix a problem. It is also costly, because every question carries a lot of unnecessary text.

With **multi-agent design**, each agent does one job well. The disease agent only thinks about disease. The irrigation agent only thinks about water. It is like a hospital. You do not want one doctor for everything. You go to the right specialist.

For this, I used **LangGraph**.

LangGraph lets us build a workflow as a **graph**. It has three main ideas.

- **State:** a shared notepad. It holds the user question, language, location, retrieved documents, and answers. Every agent can read it and add to it.
- **Nodes:** the steps or agents. Each node takes the state, does its work, and updates the state.
- **Edges:** the arrows that decide which node runs next. **Conditional edges** let the graph choose a path, for example, "if the question is about disease, go to the disease node."

Why LangGraph, and not just LangChain chains? A simple chain runs in a fixed straight line. My app needs **decisions, branches, and sometimes loops**. For example, if the weather API fails, the graph can go to a fallback node. If a question needs two agents, such as both soil and crop, the graph can run both.

LangChain still helped me with the building blocks, such as prompt templates, document loaders, text splitters, and the retriever.

Part 6: The agents one by one (3 min)

Let me explain each agent briefly.

1. Router agent. It reads the question and chooses the topic. I used the LLM with a short prompt that returns only one label, like "disease" or "irrigation". I kept the prompt short, so it is cheap and fast. If the question is unclear, it goes to a general RAG answer node.

2. Crop planning agent. It takes soil values (N, P, K, pH), temperature, humidity, and rainfall. It sends them to my **machine learning model**, which predicts the best crops. Then the LLM explains why these crops suit the conditions and when to sow them.

3. Disease agent. The farmer describes the symptoms, like yellow spots or curling leaves, and the crop name. The agent uses a trained model to find the likely disease. Then it uses **RAG** to find treatment and prevention steps from my knowledge base. The LLM writes simple advice. I also added a warning for serious cases: please meet a local agriculture officer.

4. Irrigation agent. It gets the weather forecast and the crop type. It does a **simple water balance**: water lost by evaporation and plant use, minus rainfall. If heavy rain is coming, it tells the farmer to skip irrigation. If it will be hot and dry, it suggests more frequent watering. The agent then makes a short schedule for the next few days.

5. Soil analysis agent. It checks soil values against normal ranges. For example, if pH is too low, the soil is acidic, and it may suggest lime. If nitrogen is low, it may suggest organic manure or a suitable fertiliser. These checks are rules, and the LLM only explains them. This keeps important numbers safe from LLM mistakes.

6. Climate risk agent. It reads the forecast for temperature and rainfall and checks them against thresholds. It then gives a risk level, such as low, medium, or high, for heat stress, drought, or heavy rain.

7. Answer node with RAG. For general questions, it searches the knowledge base and answers based only on what it finds.

An important design idea: wherever **numbers and rules** matter, I used code or ML. Wherever **language and explanation** matter, I used the LLM. This reduces hallucinations.

Part 7: The RAG pipeline (3 min)

Now the RAG part. RAG means Retrieval-Augmented Generation.

Why RAG? An LLM has general knowledge. But it does not know my local guides, and it may invent facts. RAG gives the LLM **trusted documents** to read before it answers.

The pipeline has two phases.

Phase 1: Building the knowledge base (done once).

1. **Collect documents.** I collected agriculture guides and crop and disease information from trusted sources, such as government and agriculture university material, in PDF and text form.
2. **Split into chunks.** Long documents are cut into small parts of about [500 to 1,000] characters, with a small overlap. A chunk should be big enough to hold one idea, but small enough to be precise.
3. **Create embeddings.** Each chunk is turned into a list of numbers using a **Hugging Face embedding model**. Chunks with similar meaning get similar numbers.
4. **Store in FAISS.** The vectors go into **FAISS**, a fast vector search library from Meta. I saved the index on disk, so I do not have to rebuild it each time.

Phase 2: Answering a question (every time).

1. The user's question is turned into a vector with the same embedding model.
2. FAISS finds the top [3 to 5] closest chunks.
3. The chunks and the question are put in a prompt for the LLM.
4. The prompt tells the LLM: **"Answer only using this context. If the answer is not in the context, say you do not know."**
5. The LLM writes the answer, and the app can show the **source** of the information.

Why did I use FAISS? It is free, fast, and runs locally, so I did not need a paid vector database.

Some small choices that helped quality:

- **Overlap in chunks**, so a sentence is not cut in the middle of an idea.
- **Metadata** such as crop name and topic on each chunk, so I can filter results.
- **Low temperature** on the LLM, so the answers stay factual.

Part 8: The machine learning part (1.5 min)

I also used classic machine learning.

For crop recommendation, I used the Kaggle crop recommendation dataset. It has soil nutrients, temperature, humidity, pH, and rainfall, and the best crop for each case. I used **Pandas** for cleaning and **NumPy** for number work. I tested a few models, and a **Random Forest** gave the best results, with an accuracy of about [__] percent. I split the data into training and test sets, so the score is honest.

Why Random Forest? It works very well on table data. It is fast. It handles different feature scales. It also shows **feature importance**, so I can tell which factor matters most.

For disease prediction, I used [a classifier trained on symptom and crop features]. The input is converted to numbers with encoding, and the model gives the most likely disease with a probability.

The model saves time and cost, because it is quick and does not need an API call. The LLM is only used to explain the result, not to guess it.

Part 9: Multilingual support (1 min)

Many farmers are more comfortable in their own language. So I added Hindi and Telugu, along with English.

I did it in two ways.

First, the user selects a language in the app. I put this language in the prompt, so the LLM replies in that language.

Second, my knowledge base is mostly in English. A Telugu question needs to find English documents. So I used a **multilingual embedding model** from Hugging Face. It puts similar meanings close together, even across languages. So a Telugu question can match an English chunk.

I also added a short instruction that the answer should use **simple words and short sentences**, which is easier for farmers.

Part 10: Caching and API optimisation (2 min)

LLM APIs have limits on how many requests you can make, and calls take time. Many farmers ask the same kinds of questions, like "which crop for black soil in kharif season?" So I added **intelligent caching**.

How it works:

- I make a **cache key** from the cleaned question, plus location, language, and key inputs. Cleaning means lower-casing and removing extra spaces.
- Before calling the LLM, I check the cache. If the key exists, I return the saved answer at once.
- If not, I call the LLM and save the result.

Some smart details:

- **Different expiry times.** Weather and forecast answers expire in a few hours, because weather changes. General farming knowledge can stay longer.
- **Caching smaller parts.** I cache things like weather API results and embeddings separately, so repeated work is avoided.
- **Short prompts and limited context.** I send only the top few chunks, not the whole document, to save tokens.
- **A small, fast model for routing,** and a stronger model only for the final answer, if needed.
- **Fallback and retry.** If the API fails or hits a limit, the app shows a friendly message or a cached answer, and does not crash.

Result: Fewer API calls, faster replies, lower cost, and better scalability when many users use the app. [If you measured it, say: "Repeated questions came back in under a second, compared to about [__] seconds."]

Part 11: Streamlit interface (1 min)

For the front end, I used **Streamlit**. It lets me build a web app using only Python, so I could focus on the AI logic.

The app has a sidebar for language and location, tabs for each feature, input forms for soil values, and a chat area for the assistant. I added **HTML and CSS** for custom styling: colours, cards, and clear fonts, so it looks clean and is easy to read.

I used **session state** in Streamlit to remember the chat history and the user's settings during a session.

I kept the design very simple, because many users are not technical.

Part 12: Demo walk-through (1 min)

Let me describe a short demo.

A farmer selects Telugu and enters the location. In the crop planning tab, they enter soil values and press a button. The app shows the top three crops, with reasons and a sowing time.

Then they go to the chat and ask, "My tomato leaves have yellow spots. What should I do?" The router sends it to the disease agent. The app shows the likely disease, the treatment steps, and the source of the information, with a note to meet an agriculture officer if it gets worse.

In the irrigation tab, they get a 5-day watering plan based on the forecast. If rain is expected on day 2, the plan says to skip irrigation that day.

Part 13: Challenges (1.5 min)

I faced a few problems.

Challenge 1: Hallucinations. The LLM sometimes gave confident but wrong tips. I fixed this with RAG, a strict "answer only from the context" prompt, rules and ML for important numbers, and a low temperature.

Challenge 2: Wrong routing. The router sometimes picked the wrong agent for mixed questions. I improved the prompt with examples, added a general fallback node, and allowed more than one agent for mixed questions.

Challenge 3: API limits and speed. Free API tiers have rate limits. Caching, shorter prompts, and retry logic fixed most of this.

Challenge 4: Poor retrieval. At first, the chunks were too big, and the results were noisy. I tested different chunk sizes and overlaps, and used metadata filters.

Challenge 5: Language quality. Replies in regional languages sometimes used difficult words. I added instructions for simple words and short sentences.

Challenge 6: Keeping secrets safe. I moved API keys into environment variables and kept them out of my code and GitHub.

Part 14: Limitations and future work (1.5 min)

I want to be honest about the limitations.

- It is an **advisory tool, not an expert replacement.** Serious cases need a human agriculture officer.
- The quality depends on my **knowledge base** and the **datasets.** The crop dataset is not specific to every district.
- **Weather forecasts** can be wrong, so risk advice can also be wrong.

- LLMs can still make mistakes, even with RAG.
- I have not tested it with **real farmers at scale.**
- Internet is needed, and some farmers have weak connections.

Future work:

- Add **leaf image disease detection** using a CNN.
- Add **voice input and output**, so farmers who cannot read can use it.
- Provide a **WhatsApp or Telegram** version, since many farmers already use these apps.
- Use **real-time soil sensor data** (IoT).
- Add **market price information** and government scheme details.
- Use a **local or smaller open-source model** for low-cost or offline use.
- Add **evaluation and monitoring**, such as answer quality checks, feedback buttons, and logs.
- **Deploy on the cloud** with proper authentication and monitoring.

Part 15: Closing (1 min)

To summarise: I built an AI assistant that helps farmers with crop planning, disease advice, irrigation, soil analysis, and climate risk, in their own language.

Technically, it uses a **multi-agent LangGraph workflow**, a **Groq LLM**, a **RAG pipeline with FAISS and Hugging Face embeddings**, **machine learning models**, and **smart caching**, all inside a **Streamlit** app.

The biggest lesson was this: an LLM alone is not enough. A good AI product mixes the LLM with **real data, trusted documents, ML, and rules**, and it also thinks about cost, speed, and safety.

Thank you. I am happy to answer any questions.

FOLLOW-UP QUESTIONS THE INTERVIEWER MAY ASK

Practise these in your own words.

1. What is RAG and why did you use it?
RAG gives the LLM trusted documents to read before it answers. I used it to reduce hallucination and to use local, reliable farming knowledge.

2. How does the RAG retrieval work?
The question is converted into an embedding, FAISS finds the most similar chunks by vector distance, and those chunks go into the prompt.

3. What are embeddings?
Lists of numbers that capture the meaning of text. Similar meanings give vectors that are close together.

4. Why FAISS?
It is free, fast, and runs locally, so no paid vector database is needed for a project of this size.

5. How did you decide the chunk size?
By testing. Chunks that were too big added noise, and chunks that were too small lost context. I used a middle size with a small overlap.

6. What is LangGraph, and how is it different from LangChain?
LangChain gives building blocks and simple chains. LangGraph builds workflows as a graph with shared state, branches, and loops, which suits agents that need decisions.

7. What is "state" in LangGraph?
A shared object that carries the question, language, retrieved text, and results between nodes.

8. What are conditional edges?
Edges that choose the next node based on the state. I used them so the router can send a question to the right agent.

9. Why multi-agent and not one prompt?
Each agent has a small, focused task, so there are fewer mistakes, lower token cost, and it is easier to test and fix.

10. How did you reduce hallucination?
RAG with an "answer only from context" rule, ML and rules for numbers, low temperature, showing sources, and a "please consult an expert" note.

11. How does your caching work, and what are the risks?
It stores answers by a key made from the cleaned question, location, and language, with an expiry time. Risks are stale answers (so I use short expiry for weather) and cache poisoning or private data in cache (so I avoid storing personal details and control access).

12. What if the Groq API fails or hits a rate limit?
The app retries after a short delay, then uses a cached answer if available, or shows a friendly message. It does not crash.

13. Why Groq?
It serves open-source models with very fast responses and has a free tier, which is good for a student project and for a smooth chat experience.

14. How did you evaluate the system?
For ML, I used a test set with accuracy, precision, and recall. For RAG and LLM answers, I tested a set of sample questions and checked whether the answers were correct, grounded in the sources, and in the right language. In future, I would add automatic evaluation and user feedback.

15. How did you handle multilingual questions?
Language selection in the prompt for the answer, and a multilingual embedding model so questions in Telugu or Hindi can retrieve English documents.

16. Why Random Forest for crop recommendation?
It works well on table data, handles different scales, resists overfitting better than a single tree, and shows feature importance.

17. How do you scale this to thousands of users?
Deploy behind a proper web server, use a shared cache like Redis, move FAISS to a managed vector database if needed, use async calls and queues, and monitor cost and latency.

18. What is the difference between prompt engineering, RAG, and fine-tuning?
Prompt engineering changes the instructions. RAG adds outside knowledge at question time. Fine-tuning changes the model weights. I chose RAG because knowledge changes often and I wanted sources.

19. What would you do differently?
Add leaf image detection, voice support, automatic evaluation, and test with real farmers.

20. Is it really production-ready?
Be honest: "It is deployment-ready. It has caching, error handling, and secret management. For real production I would add authentication, monitoring, logging, load testing, and review of the answers by agriculture experts."

Bonus: Security and privacy questions (very useful for your Deloitte interview)

Your project uses LLMs, RAG, agents, and API keys, which are exactly the topics a cyber security interviewer likes. Prepare these.

21. How did you protect your API keys?
I stored them in environment variables or Streamlit secrets and kept them out of the code and GitHub. If a key leaks, I can revoke it and create a new one. In production, I would use a secrets manager and rotate keys.

22. What is prompt injection, and could it affect your app?
It is when a user, or hidden text in a document, tells the LLM to ignore its rules. In my app, a

user could type "ignore previous instructions", or a bad document in the knowledge base could carry hidden instructions. To reduce the risk, I keep system instructions separate from user text, only use trusted documents, filter the input and output, and give the agents no dangerous tools.

23. What is the risk of "excessive agency" in agents?
If an agent has too many permissions, a trick could make it do harmful actions. My agents only read data and give advice. They cannot change systems or send data out. In general, use least privilege and human approval for risky actions.

24. Can the cache cause a security problem?
Yes. A cache can store private data, or serve one user's answer to another. A poisoned cache can serve wrong advice. So I key the cache carefully, avoid storing personal details, set expiry times, and in production I would add access control.

25. What about user data privacy?
Location and farm details are personal data. I would collect only what is needed, tell users clearly how it is used, avoid storing it longer than needed, and follow rules like India's DPDP Act. Note that messages sent to an external LLM API leave my system, so I should not send unnecessary personal data.

26. How would you attack-test this application?
Try prompt injection and jailbreak inputs, very long inputs, strange characters, and wrong file types. Test whether the app leaks its system prompt or keys, whether the RAG retrieval can be tricked, and whether rate limiting works. The OWASP Top 10 for LLM Applications is a good checklist.

27. What are the risks of third-party models and libraries?
Supply chain risk. A bad model or package could contain malicious code. So I use trusted sources, pin versions, scan dependencies, and avoid loading unknown model files.

Quick code to remember

You may be asked to explain small pieces of code. Understand these.

A small LangGraph router:

```
from langgraph.graph import StateGraph, END
from typing import TypedDict

class State(TypedDict):
    question: str
    topic: str
    answer: str

def router(state):
```

```
    # LLM returns one label: crop / disease / irrigation / soil / climate / general
    return {"topic": classify(state["question"])}

def disease_node(state):
    return {"answer": run_disease_agent(state["question"])}

def irrigation_node(state):
    return {"answer": run_irrigation_agent(state["question"])}

graph = StateGraph(State)
graph.add_node("router", router)
graph.add_node("disease", disease_node)
graph.add_node("irrigation", irrigation_node)
graph.set_entry_point("router")
graph.add_conditional_edges("router", lambda s: s["topic"],
                            {"disease": "disease", "irrigation": "irrigation"})
graph.add_edge("disease", END)
graph.add_edge("irrigation", END)
app = graph.compile()
```

A simple cache idea:

```
import hashlib, time
cache = {}

def make_key(q, location, lang):
    text = f"{q.strip().lower()}|{location}|{lang}"
    return hashlib.sha256(text.encode()).hexdigest()

def get_answer(q, location, lang, ttl=3600):
    key = make_key(q, location, lang)
    if key in cache and time.time() - cache[key]["time"] < ttl:
        return cache[key]["answer"]
    answer = call_llm(q, location, lang)
    cache[key] = {"answer": answer, "time": time.time()}
    return answer
```

A simple FAISS RAG idea:

```
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
```

```
emb = HuggingFaceEmbeddings(model_name="sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2")
db = FAISS.from_documents(chunks, emb)
db.save_local("faiss_index")

retriever = db.as_retriever(search_kwargs={"k": 4})
docs = retriever.invoke("How to treat leaf spot in tomato?")
```

If your real code is different, explain your real version. These are only memory aids.

Campus Ambassador at Aenexz

Opening (15 seconds)

"I worked as a Campus Ambassador for Aenexz for one month. Aenexz provides [technical courses and training] for students. My job was to represent the company in my college and connect students with these courses."

What I did (55 seconds)

"In one month, I reached students across my whole college.

I shared information about the courses through [classroom visits, WhatsApp groups, and social media]. I explained what each course teaches and how it can help in a career.

Many students had questions, like 'Is this useful for me?' or 'Where do I start?' I answered them in simple words and helped them choose the right course.

As a result, **17 students purchased the courses.**"

What I learned (40 seconds)

"This role helped me build three skills.

Communication: I learned to explain technical topics clearly to students from different branches.

Leadership: I took the responsibility on my own. I planned how to reach students and followed up with them.

Teamwork: I worked with the Aenexz team and with my friends to spread the word and solve problems."

Closing (10 seconds)

"In just one month, I learned how to convince people and work toward a target. I will use these skills with teammates and clients at Deloitte."

Follow-up questions and answers

1. How did you reach the whole college?
"I used [WhatsApp groups, class announcements, Instagram, and friends in different years]. I made sure each department got the information." (Use your real methods.)

2. How did you convince 17 people to buy?
"I first understood what the student wanted. Then I explained how the course could help with that goal, such as skills for internships or placements. I did not push. I only explained the value and answered their doubts."

3. What was the conversion rate?
Only answer if you know it. For example, if you reached 500 students and 17 bought, say "around 3 to 4 percent." If you don't know, say: "I did not track it exactly, but 17 students purchased."

4. What was the biggest challenge?
"Some students did not trust paid courses. I answered by sharing the course content and the benefits, and by being honest about what the course offers."

5. What would you do differently?
"I would track my numbers from the start, and I would organise a free demo session to build trust faster."

Member of IAENG

"I am a member of IAENG, which stands for the International Association of Engineers. It is a global professional organization for engineers, computer scientists, and researchers. I joined it because I want to keep learning beyond my college syllabus and stay connected with the wider technical community. As a member, I get access to [conferences, journals, and research publications] in areas like computer science and data engineering. This helps me understand new trends in fields like artificial intelligence and data science. It also shows my interest in

professional growth and learning new technology on my own. Being part of an international community reminds me that technology keeps changing, so I must keep updating my skills. I believe this habit will help me in a fast-changing field like cyber security, where continuous learning is very important."

Fill in your real details

Replace the [brackets] with what is true for you: when you joined, your membership type (student or regular), and whether you have used any benefits. If you have read papers, attended a webinar, or joined a conference through IAENG, add one line about it. A real example is much stronger than a general statement.

If the interviewer asks a follow-up

"What do you get from this membership?"
"I get access to [technical publications, conferences, and a professional network]. I use it to learn about new research and trends."

"Is this an achievement or an award?"
Be honest: "No, it is a professional membership. I see it as a sign of my interest in continuous learning, not as an award."

"Have you attended any event or published any paper?"
Answer only with what you actually did. If you have not, say: "Not yet, but I plan to read their publications and, in the future, submit my own work."

Social Awareness Initiative

"In November 2024, I conducted an awareness program on the empowerment of SC and ST communities and the prevention of atrocities. It was held in Chimakurthy village, and it reached around 1,500 students from Sri Chaitanya and QIS.

I chose this topic because discrimination still exists in society, and many young people do not know their rights or the laws that protect them.

In my session, I explained in simple words what discrimination is and why it is wrong. I told them that the Constitution gives equal rights to everyone, and that the SC/ST (Prevention of Atrocities) Act protects these communities. I also told them where to go for help, such as the police. Then I spoke about empowerment, which means equal access to education, scholarships, and opportunities, so that every person can grow.

Speaking to such a large and mixed audience was a big learning experience. It taught me public speaking, planning, teamwork, and how to handle a sensitive topic with respect.

It also showed me that a technical student should use his knowledge and time to serve society. I want to bring this sense of responsibility to my work, and also to security awareness. Many cyber attacks succeed because people do not know the risks, and I know how to explain a serious topic simply to a large group."

Research Paper

"I published a research paper titled **'Plate Dispenser Using Face Recognition to Prevent Food Wastage in Hostel Mess'** at the International Conference on Machine Learning for IT Infrastructure and Big Data, ICMLITIBD-26.

The problem: In hostel messes, a lot of food is wasted. Day scholars enter without permission, and some people collect food again and again. The old methods, like manual checking, ID cards, and verbal checking, are slow, make mistakes, and are easy to misuse.

My solution: I built an automatic plate dispensing system that uses face recognition. A camera captures the person's face. The system detects the face with OpenCV and compares it with a trained dataset of hostel students and non-hostelers. If the person is an authorised hostel student and has not taken food in this meal, the system opens a lock and gives one plate. Otherwise, it stays locked.

How it works: A Raspberry Pi does the image processing and face recognition. An Arduino controls the hardware through a relay module, which operates a solenoid lock. The system also records who has already been served, so nobody gets a second plate in the same meal.

Results: I used a dataset of 800 individuals for training and validation, and I tested the system with live images. The results showed that it reduced food wastage, stopped unauthorised access, and improved transparency and efficiency. [Add your accuracy: 'The accuracy was about __ percent.']

Why it matters: It is low cost and easy to scale. In future, we can add cloud storage and a mobile app to monitor it.

From this work, I learned computer vision, hardware integration, and how to write and present research."

Follow-up questions and answers

1. Why face recognition and not RFID or fingerprint?
"RFID cards can be shared or lent. Fingerprint needs touching a sensor, which is slow and less hygienic. Face recognition is contactless, quick, and cannot be easily passed to a friend."

2. What is the difference between face detection and face recognition?
"Detection finds that there is a face in the image. Recognition finds whose face it is."

3. How does the system stop a second plate in the same meal?
"After a person is served, the system saves their ID with a time stamp for that meal. If the same person comes again, the system finds the record and keeps the lock closed. The record resets for the next meal."

4. Why Raspberry Pi and Arduino together?
"The Raspberry Pi is a small computer, good for running OpenCV and the recognition model. The Arduino is simple and reliable for controlling the relay and the lock. Each does what it is best at."

5. What does the relay do?
"The Arduino gives only a small signal. A solenoid lock needs more power. The relay works like an electric switch that lets the small signal control the higher-power lock safely."

6. What are the limitations?
"Poor light, masks, or big changes in appearance can lower accuracy. The dataset has 800 people, so a bigger hostel needs more data. A new student must be registered first."

7. How can the system be improved?
"Add liveness detection, cloud integration, a mobile dashboard, and a stronger deep learning model."

Bonus: security and privacy questions (important for your Deloitte interview)

Face recognition handles **biometric data**, which is very sensitive. A cyber security interviewer is likely to ask these.

8. Can someone fool the system with a photo or video?
"Yes. This is called a **spoofing or presentation attack**. A printed photo or a phone screen could trick a basic system. The fix is **liveness detection**, such as blink detection, head movement, depth sensing, or an infrared camera."

9. How would you protect the students' face data?
"Store face embeddings (numerical templates) instead of raw photos, encrypt the data, limit access, and delete the data when a student leaves. Take written consent and tell students how their data is used."

10. What law applies?
"In India, the Digital Personal Data Protection Act, 2023 requires consent and purpose limitation for personal data. In Europe, GDPR treats biometric data as a special category."

11. What happens if the face database is stolen?
"Unlike a password, a face cannot be changed. So a leak is very serious. This is why I would encrypt the data, use access control, keep logs, and store templates rather than photos."

12. What are false accept and false reject?
"A false accept means the system lets in a wrong person, which is a security failure. A false reject means a real student is denied, which is an inconvenience. We tune the threshold to balance both."

13. How would you secure the Raspberry Pi?
"Change default passwords, disable unused services, update the software, use SSH keys, encrypt the storage, and protect the device physically so nobody can open it or unplug it."

14. Is face recognition biased?
"It can be. Accuracy can differ across skin tones, lighting, and age if the training data is not balanced. I would test on all groups and keep a manual backup for people the system cannot recognise."